
ASSERT: AUDITING IMPLICIT ASSUMPTIONS IN LLM-BASED CODING OUTPUTS

En-Chao Liu Ava Zheer Wang Binyue Deng

Georgia Institute of Technology

eliu354@gatech.edu, zwang3657@gatech.edu, bdeng72@gatech.edu

1 COLLABORATION AND TEAM MEMBER CONTRIBUTIONS

All team members contributed to the overall project direction, including discussions on the system architecture, experimental setup, monitor comparison design, and evaluation procedure. En-Chao Liu worked on the general coding pipeline, OpenAI Batch API integration, human-review JSON artifacts, the revised evaluation protocol, and the Breakpoint 0 auditor ablation. Ava Zheer Wang contributed to frontend and backend prompt-category design, task construction, experimental pipeline planning, and multi-agent monitor design. Binyue Deng contributed to data-processing and DevOps prompt-category design, redaction-task construction, and the assumption-monitor framework implementing the inline, external, and multi agent monitor variants.

2 ABSTRACT

LLM-based coding agents often resolve underspecified user requests by making implicit implementation assumptions. These assumptions can affect data formats, defaults, persistence, error handling, edge cases, and user interface behavior, yet they are rarely exposed for user inspection. We introduce ASSERT, a framework for auditing implicit assumptions in LLM-based coding outputs. Given a redacted task specification and a coding agent’s generated solution, ASSERT extracts structured assumptions with categories, evidence, and rationales. We compare three monitoring strategies: inline self-reporting, full-context external auditing, and a multi-agent monitor with decomposition, extraction, merging, and verification. We evaluate these monitors using both quality metrics, which measure whether extracted assumptions are correct, overlapping, wrong, or hallucinated, and value metrics, which distinguish targeted recovery of predefined golden assumptions from discovery of additional valid assumptions. Across 60 prompt-output records from five coding domains, the full-context external auditor achieves the strongest overall performance, with 93.7% precision, 46.7% targeted recall, and 500 discovered assumptions. The multi-agent monitor extracts more assumptions per task but suffers from higher overlap, suggesting that decomposition can weaken prompt-output alignment when outputs still fit within a single model context window. Finally, about 58% of correct assumptions are discoveries outside the golden set, showing that assumption monitors are not only retrieval tools for known missing requirements but also diagnostic tools for surfacing unanticipated design decisions.

3 INTRODUCTION

LLM-based coding agents are increasingly used to turn natural-language instructions into software artifacts. Unlike earlier autocomplete-style tools, modern agents can generate multi-file solutions, modify code, and make implementation decisions with limited user intervention. However, real software requirements are often incomplete: a user may ask an agent to “build a URL shortener” or “deduplicate a CSV” without specifying storage choices, input formats, duplicate-handling behavior, error policies, or edge cases.

When such details are missing, an ideal agent might ask clarification questions. In practice, coding agents often proceed directly and fill in the missing information themselves. The generated output may look complete, but it can contain many implicit assumptions that were never confirmed by the user. Some assumptions may be reasonable defaults, while others may be incorrect, unsafe, or

inconsistent with the user’s intended requirements. Because these assumptions are embedded in the generated code or plan, they are difficult for users to notice and inspect.

Existing code-agent evaluations have largely focused on functional correctness or clarification behavior before generation. In contrast, we study a complementary post-generation diagnostic problem: after an agent has already produced a solution, can we audit the output and recover the assumptions behind it? We propose ASSERT, a framework for surfacing hidden assumptions in coding-agent outputs under underspecified requirements. Given a redacted task specification and the agent’s generated output, ASSERT extracts structured assumptions with categories, evidence, and rationales. We compare three monitoring strategies: inline self-reporting, full-context external auditing, and a multi-agent monitor. We evaluate these monitors using both quality labels and value-oriented recall/discovery metrics, allowing us to distinguish assumptions that recover predefined missing requirements from additional valid assumptions discovered in the generated output.

Our experiments show that assumption monitoring reveals many hidden design decisions made by coding agents. Across 60 prompt-output records from five coding domains, a full-context external auditor achieves the best overall performance, with 93.7% precision, 46.7% targeted recall, and 500 discovered assumptions. Our analysis suggests that assumption extraction is fundamentally a prompt-output alignment task: the monitor must determine not only whether a decision appears in the generated code, but also whether it was already specified by the user. This explains why full-context auditing performs strongly, while chunk-level multi-agent decomposition can increase overlap by weakening the contrast between prompt-stated constraints and agent-made decisions.

Our contributions are threefold:

1. We formulate post-generation assumption auditing as a diagnostic task for LLM-based coding agents under underspecified requirements.
2. We introduce ASSERT, a structured framework that extracts assumptions with categories, evidence, and rationales from prompt-output pairs.
3. We construct a 60-record benchmark across five coding domains and show that full-context external auditing achieves the strongest overall performance, while approximately 58% of correct assumptions are discoveries outside the predefined golden set.

4 RELATED WORK

Early evaluation of LLM-based code generation has primarily focused on functional correctness. Benchmarks such as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) evaluate whether models can synthesize programs that pass unit tests from relatively clear natural-language specifications. More realistic software-engineering benchmarks such as SWE-Bench evaluate repository-level issue resolution, but still primarily score final task success. Recent work argues that such final-outcome metrics are insufficient for agentic systems because they provide limited insight into intermediate decisions and failure causes. For example, Agent-as-a-Judge proposes using agentic evaluators for code-generation agents and introduces DevAI, a benchmark of realistic AI development tasks with hierarchical requirements (Zhuge et al., 2025). Our work shares the motivation of richer agent evaluation, but focuses specifically on surfacing the implicit assumptions that shape generated code rather than only judging requirement satisfaction or final correctness.

A growing line of work studies ambiguity and underspecification in code generation. HumanEvalComm (Wu & Fard, 2025) introduces communication-oriented code-generation tasks with inconsistency, ambiguity, and incompleteness, while ClarEval (Li et al., 2026) evaluates clarification skills under missing goals, missing premises, and ambiguous terminology. Ambig-SWE extends this direction to realistic software-engineering tasks by evaluating agents on detecting underspecification, asking targeted clarification questions, and using the resulting interaction to improve performance (Vijayvargiya et al., 2026). These works show that underspecification is a central challenge for coding agents. However, they mainly study the pre-generation setting: whether the agent recognizes missing information and asks a question before acting. Our work studies a complementary post-generation setting: when an agent does not clarify and has already produced a solution, can we audit the output and recover the assumptions it made?

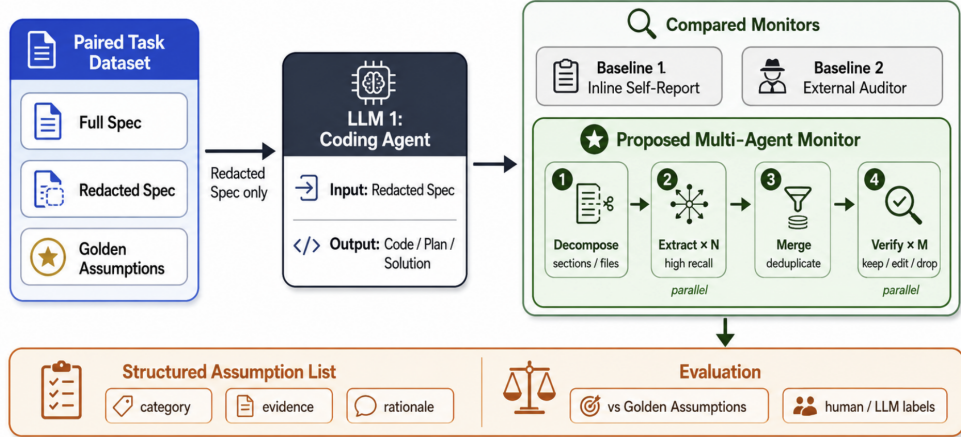


Figure 1: Overview of the ASSERT pipeline. Each paired task contains a full specification, a redacted specification, and golden assumptions. The coding agent only receives the redacted specification and generates a code output. Different monitors then audit the prompt-output pair and produce structured assumptions with categories, evidence, and rationales. The extracted assumptions are evaluated against golden assumptions and human/LLM labels.

Our project is also related to agent observability, assumption-aware reasoning, and verification. AgentTrace constructs causal graphs from multi-agent execution logs, traces backward from error manifestations, and ranks candidate root causes for post-hoc failure diagnosis (Wang, 2026). PCE converts fragmented assumptions latent in LLM reasoning traces into a structured decision tree for uncertainty-aware embodied planning (Seo et al., 2026). These works demonstrate the value of making intermediate agent behavior more inspectable, but they target different objects: AgentTrace diagnoses failures from execution traces, while PCE uses assumptions to guide action selection. In contrast, ASSERT audits prompt-output pairs in coding workflows, aiming to surface hidden implementation assumptions even when the generated solution appears complete and no explicit failure has yet occurred. This also connects to verification and hallucination-detection methods such as Chain-of-Verification and SelfCheckGPT, which distinguish grounded outputs from unsupported claims (Manakul et al., 2023). However, rather than verifying final answers alone, we treat assumption monitoring as a first-class auditing task and evaluate monitors both as retrieval systems for predefined missing requirements and as discovery tools for unanticipated valid design decisions.

5 APPROACH

Figure 1 summarizes the overall ASSERT pipeline. Our goal is to audit the hidden assumptions made by a coding agent when it generates a solution from an underspecified task prompt. Each task is represented as a triple

$$\tau = (s_{\text{full}}, s_{\text{redacted}}, G),$$

where s_{full} is the complete task specification, s_{redacted} is the redacted specification shown to the coding agent, and $G = \{g_1, \dots, g_m\}$ is the set of golden assumptions corresponding to removed constraints or decision dimensions. The coding agent only observes s_{redacted} and produces an output y consisting of generated code. A monitor then takes the prompt-output pair and extracts a set of assumptions:

$$A = M(s_{\text{redacted}}, y) = \{a_1, \dots, a_n\}.$$

Each extracted assumption contains four fields: content, category, evidence, and rationale.

5.1 TASK SETUP

The task setup is designed to expose the assumption space created by underspecified coding prompts. For each task, we start from a full specification and create a redacted version by removing important

requirements. The removed information is recorded as golden assumptions. The redacted prompt is given to the coding agent, while the full specification and golden assumptions are only used for evaluation.

We use two redaction levels. In L1, the overall task structure is preserved, but concrete constraints are removed, such as storage choices, default values, input formats, output formats, or edge-case behavior. This level tests whether a monitor can recover specific missing requirements. In L2, the prompt is reduced to a minimal high-level goal, such as “build a URL shortener.” This level tests whether a monitor can identify broader decision dimensions that the agent must resolve on its own.

5.2 BASELINES

We compare our proposed monitor against two simple baselines.

Inline self-reporting The first baseline asks the coding agent to generate the solution and list its own assumptions in the same response. This method uses a single LLM call and is inexpensive. However, because the same model both makes the assumptions and reports them, it may fail to notice its own blind spots.

External auditing The second baseline separates generation from auditing. First, the coding agent generates a solution from the redacted prompt. Then, a separate monitor model receives both the redacted prompt and the generated output, and extracts assumptions in the same structured format. This baseline allows the auditor to use a stronger model than the coding agent and tests whether an independent monitor can surface assumptions that the coding agent does not report itself.

5.3 MULTI-AGENT MONITOR

Our main approach is a four-stage multi-agent monitor that decomposes long coding outputs and audits each part separately. The design is motivated by the observation that a single-pass monitor may overlook assumptions buried in a specific function, configuration block, or UI component. By decomposing the output and auditing each part independently, the pipeline aims to improve recall; by adding merging and verification, it controls precision.

The first stage is the **decomposer**, which splits the coding output into chunks by deliverable. When the output contains markdown section headers, we split on them using a rule-based parser; otherwise, we fall back to an LLM-based decomposer. Outputs shorter than 2,500 characters are not split and are treated as a single chunk.

In the second stage, **extractor** agents audit each chunk independently, receiving the full redacted prompt alongside the assigned chunk. Extractors operate in high-recall mode, proposing 8–15 candidate assumptions per chunk. Before including any candidate, each extractor performs an anti-overlap pre-check by searching the prompt for verbatim or near-paraphrase matches; candidates that restate prompt content are excluded at this stage.

The third stage is the **merger**, which receives all candidates grouped by chunk, deduplicates semantically similar entries, and marks cross-cutting assumptions that span multiple chunks. The merger does not introduce new assumptions; it only consolidates existing candidates.

In the final stage, **verifier** agents evaluate each merged candidate on four criteria: (1) whether the described decision exists in the code, (2) whether the prompt already specified it, (3) whether the description accurately reflects the code’s behavior, and (4) whether the decision is significant rather than trivial. Candidates passing all four checks are kept; those with inaccurate descriptions are edited with corrected content; all others are dropped.

5.4 STRUCTURED ASSUMPTION FORMAT

All monitors output assumptions in the same structured format. The *content* field states the implicit assumption. The *category* field identifies the type of decision, such as persistence, data format, interface, defaults, edge cases, behavior, or UI. The *evidence* field points to the part of the generated output that supports the assumption. The *rationale* field explains why the assumption is not directly stated in the redacted prompt.

Table 1: Dataset composition: 22 tasks across 5 coding domains.

Domain	Example Tasks	# Tasks	# Records
Backend API	auth-service, url-shortener, file-upload	5	15
Frontend Web	todo-app, kanban-board, markdown-editor	5	14
Data Processing	csv-etl-pipeline, log-aggregator, json-diff	5	11
DevOps/Infra	ci-cd-pipeline, k8s-manifest, terraform-vpc	4	10
General Coding	regex-engine, lru-cache, expression-evaluator	3	10
Total		22	60

This shared format makes outputs from different monitors comparable. It also supports downstream evaluation: evidence helps determine whether an assumption is grounded in the generated output, while the rationale helps distinguish true hidden assumptions from information already specified in the prompt.

6 EXPERIMENTS

6.1 DATA

We construct a benchmark of 22 coding tasks spanning five domains: backend API (5 tasks), frontend web (5), data processing (5), DevOps/infrastructure (4), and general coding (3). Table 1 summarizes the dataset composition. Each task consists of a detailed natural-language specification (~260 words), a deliberately underspecified redacted version simulating a realistic user prompt, and a set of golden assumptions — the constraints removed during redaction that define the design decisions we expect the monitor to recover.

Task specifications, redacted variants, and golden assumption annotations were generated using an LLM and subsequently validated by the authors to ensure task realism, redaction plausibility, and golden completeness.

We apply two redaction strategies to produce distinct evaluation settings, which yields 38 L1 records and 22 L2 records, totaling 60 evaluation records.

Under **L1 (structural redaction)**, specific values are removed while the document structure is preserved. For example, “Short codes are 6 lowercase alphanumeric characters generated via SHA-256 truncation” becomes “Short codes use [unspecified format],” and the golden assumptions are the removed values.

Under **L2 (minimal redaction)**, the entire specification is condensed to a one-sentence goal such as “Build a URL shortener web application,” and the golden assumptions are high-level decision dimensions (e.g., “storage mechanism must be chosen”).

6.2 EVALUATION FRAMEWORK

We design a two-layer evaluation framework that separates *quality assessment* from *value assessment*, addressing a limitation we identify in traditional recall-only evaluation.

Quality assessment. An LLM judge (GPT-4.1) classifies each extracted assumption into one of four categories. An assumption is *correct* if it accurately describes a real design decision present in the code that the prompt did not specify — this is the desired outcome. It is *overlap* if the decision is real but the prompt already stated it, meaning the monitor is merely restating the specification. It is *wrong* if the monitor misdescribes the code’s actual behavior (e.g., claiming “uses MD5” when the code uses SHA-256). Finally, it is *hallucination* if the described decision does not exist in the code at all. From these labels we compute precision (correct / total), along with overlap, wrong, and hallucination rates.

Value assessment. For each assumption labeled *correct*, a second LLM judge determines whether it semantically matches any entry in the golden set. Matches count as *targeted hits*, while non-

Table 2: Main results across 60 tasks (L1 + L2 combined). Best per column in **bold**.

Method	Total	Avg/task	Prec.	Overlap	Halluc.	Tgt. Recall	Discovery
Inline	381	6.3	0.738	22.6%	0.0%	0.199	159
External	920	15.3	0.937	5.5%	0.1%	0.467	500
Multi-Agent	1056	17.6	0.792	17.1%	0.0%	0.440	481

Table 3: Performance by redaction level.

Method	Level	Prec.	Overlap	Tgt. Recall	Discovery
Inline	L1	0.616	33.6%	0.147	112
	L2	0.969	1.5%	0.266	47
External	L1	0.907	8.3%	0.382	399
	L2	0.994	0.3%	0.572	101
Multi-Agent	L1	0.719	24.7%	0.343	365
	L2	0.938	2.0%	0.558	116

matches count as *discoveries*: legitimate design decisions not anticipated by the golden set. We report targeted recall (unique golden entries matched / total golden entries) and discovery count. Since golden sets are necessarily incomplete, as a coding agent makes far more decisions than any annotator can enumerate, neither metric alone captures monitor utility. Together they distinguish retrieval capability from discovery capability.

6.3 EXPERIMENTAL SETUP

Methods. We compare three monitor architectures. **Inline** is the simplest baseline, where the coding agent generates code and lists its own assumptions in a single call. **External** uses a separate, stronger LLM that audits the code with full visibility over the prompt and output in one context window. **Multi-Agent** is our proposed 4-step pipeline (Decompose, Extract, Merge, Verify), described in detail in Section 5.

Models. We use GPT-4o as the coding agent across all methods to ensure fair comparison. The External monitor and Multi-Agent Extractor/Merger use GPT-4.1; the Multi-Agent Verifier uses GPT-4o-mini. All evaluations use GPT-4.1 as the LLM judge.

6.4 RESULTS

6.4.1 OVERALL PERFORMANCE

Table 2 presents the main results across all 60 tasks.

The External method achieves the strongest overall performance: the highest precision (0.937), the highest targeted recall (0.467), and the most discoveries (500), while maintaining the lowest overlap rate (5.5%). Multi-Agent produces the most assumptions per task (17.6 vs. 15.3) but at lower precision (0.792), with the gap driven primarily by elevated overlap (17.1% vs. 5.5%). Despite this, Multi-Agent’s discovery count (481) is within 4% of External’s, indicating comparable capability at surfacing novel design decisions. Hallucination is effectively zero across all methods ($\leq 0.1\%$), confirming that at the GPT-4o/4.1 capability level, fabrication is not the bottleneck for assumption extraction.

6.4.2 EFFECT OF REDACTION LEVEL

Table 3 stratifies performance by redaction level, revealing a sharp contrast between L1 and L2.

Under L2, all methods converge to $>93\%$ precision and $<2\%$ overlap. A single-sentence prompt provides almost no content for the monitor to mistakenly echo, making overlap discrimination trivial.

Table 4: Targeted recall by domain (L1 + L2 combined).

Domain	Inline	External	Multi-Agent
Backend API	0.18	0.48	0.44
Frontend Web	0.17	0.46	0.42
Data Processing	0.18	0.34	0.38
DevOps/Infra	0.30	0.61	0.53
General Coding	0.17	0.47	0.46

Table 5: Breakdown of correct assumptions into targeted hits and discoveries.

Method	Total Correct	Targeted (%)	Discovery (%)	Ratio D/T
Inline	281	122 (43%)	159 (57%)	1.30×
External	862	362 (42%)	500 (58%)	1.38×
Multi-Agent	836	355 (42%)	481 (58%)	1.35×

Under L1, precision drops substantially — most severely for Inline (−35.3pp) and least for External (−8.7pp). This pattern suggests that full-context visibility is the key factor in overlap avoidance: External sees the entire prompt and output together, enabling precise comparison, whereas Inline and Multi-Agent each face structural limitations in maintaining this contrast. L1 is therefore the more discriminating and practically relevant evaluation setting, as it mirrors real-world prompts that contain partial but incomplete specifications.

6.4.3 CROSS-DOMAIN GENERALIZATION

Table 4 reports targeted recall across five coding domains.

Performance is broadly consistent across domains, with all methods maintaining their relative ordering and a maximum within-method spread of ≤ 0.14 in targeted recall. One notable exception is data processing, where Multi-Agent is the only domain in which it outperforms External (0.38 vs. 0.34). Data processing tasks typically produce long, logic-dense single-file code where chunk-level decomposition may provide more focused auditing than a single full-context pass. DevOps/infrastructure tasks exhibit the highest recall across all methods, likely because infrastructure code (Dockerfiles, CI/CD pipelines, Terraform configurations) contains more explicit and identifiable decision points.

6.5 ANALYSIS

6.5.1 DISCOVERY DOMINATES TARGETED RETRIEVAL

Table 5 decomposes correct assumptions into targeted hits (matching the golden set) and discoveries (absent from the golden set).

Across all three methods, approximately 58% of correct assumptions are discoveries. This ratio is stable across methods (57–58%), indicating it reflects a property of the task itself: golden sets, however constructed, capture less than half of the design decisions a coding agent makes. An evaluation based solely on targeted recall would miss more than half of the monitor’s output, underestimating its practical value. This validates the dual-track metric design and establishes that assumption monitors function primarily as discovery instruments rather than checklist verifiers.

6.5.2 ATTENTION DILUTION IN MULTI-AGENT DECOMPOSITION

The Multi-Agent approach exhibits 17.1% overlap compared to External’s 5.5%. We inspected 10 randomly sampled overlap cases; all followed the same pattern: the per-chunk Extractor re-stated a value the prompt explicitly specified. For instance, the prompt “emails stored lowercased” produced the extraction “all email addresses are stored in lowercase form”; “SQLite file jobs.db” produced “uses SQLite as storage with filename jobs.db.”

We attribute this to *attention dilution*: each Extractor receives the full prompt alongside its chunk, but concentrates on the code, treating prompt-stated values visible in the code as agent decisions. External avoids this by processing the full prompt and output in a single context window, maintaining stronger prompt-output contrast.

Two prompt-level mitigations were applied: a verbatim prompt search before extraction, and stricter overlap detection in the Verifier. These reduced overlap from 20.3% to 17.1% and increased discovery count from 213 to 481 (+126%), as the original Verifier had been over-aggressively dropping legitimate assumptions. Overlap nonetheless remains $3\times$ higher than External, confirming that attention dilution is a structural cost of decomposition, not a prompt engineering deficiency.

6.5.3 WHY EXTERNAL WINS

The dominance of the single-LLM baseline is notable, as multi-agent architectures have generally shown improvements in complex reasoning and generation tasks. The distinction is that assumption extraction is a *prompt-output alignment task*: each candidate requires simultaneous assessment of whether the decision exists in the code and whether the prompt already specified it. A single context window containing both artifacts naturally supports this comparison; decomposition weakens it.

In our benchmark, outputs average 3,000–8,000 characters, well within the model’s context capacity. The multi-agent tradeoff of alignment quality for per-chunk depth does not pay off at this scale. For substantially longer outputs where single-model attention over long contexts degrades, decomposition is expected to become advantageous.

7 CONCLUSION

We introduced assumption extraction as a new task for auditing code agent outputs, addressing the gap between an agent’s implicit design decisions and the user’s ability to verify them. Our benchmark of 22 tasks across 5 coding domains, with a dual-level redaction scheme yielding 60 evaluation records, provides the first systematic testbed for this task. We proposed a two-layer evaluation framework that separates quality assessment from value assessment through a novel discovery metric, and compared three monitor architectures of increasing complexity.

Our experiments on 2,357 extracted assumptions reveal several key insights. First, a single strong-LLM auditor with full-context visibility (External) is a formidable baseline, achieving 93.7% precision and 46.7% targeted recall, outperforming our more complex multi-agent pipeline on most metrics. This is because assumption extraction is fundamentally a prompt-output alignment task that benefits from maintaining both artifacts in a single context window. Second, multi-agent decomposition introduces an attention dilution effect, where per-chunk extractors lose the global contrast needed to distinguish prompt-stated constraints from agent-decided choices, resulting in $3\times$ higher overlap. Third, across all methods approximately 58% of correctly extracted assumptions are *discoveries*, i.e., design decisions not anticipated by the golden set, validating both the coverage problem and our dual-track evaluation design.

These findings carry practical implications. Practitioners deploying assumption monitors should default to single-LLM auditing unless output length exceeds the model’s effective context window. Researchers evaluating such systems should adopt dual-track metrics, as targeted recall alone underestimates the monitor’s value by more than half. The attention dilution phenomenon we identify in multi-agent decomposition warrants further investigation in other prompt-grounded analysis tasks.

Limitations. Our evaluation relies on LLM-based judges; human cross-validation is planned. The benchmark covers only coding tasks, and our taxonomy lacks a *contradiction* label for cases where code violates prompt requirements, though this was not observed with GPT-4o as the coding agent. Future work includes a refinement loop feeding approved assumptions back to improve agent outputs, extension to non-coding domains, and hybrid architectures that combine full-context auditing with targeted chunk-level extraction for long outputs.

REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Jialin Li, Yuan Wu, and Yi Chang. Clareval: A benchmark for evaluating clarification skills of code agents under ambiguous instructions. *arXiv preprint arXiv:2603.00187*, 2026. URL <https://arxiv.org/abs/2603.00187>.
- Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *arXiv preprint arXiv:2303.08896*, 2023. URL <https://arxiv.org/abs/2303.08896>.
- SeungWon Seo, SooBin Lim, Seongrae Noh, Haneul Kim, and HyeongYeop Kang. From assumptions to actions: Turning llm reasoning into uncertainty-aware planning for embodied agents. In *International Conference on Learning Representations*, 2026.
- Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. Ambig-swe: Interactive agents to overcome underspecificity in software engineering. In *International Conference on Learning Representations*, 2026.
- Zhaohui Geoffrey Wang. Agenttrace: Causal graph tracing for root cause analysis in deployed multi-agent systems. In *ICLR Workshop on Agents in the Wild*, 2026.
- Jie J. W. Wu and Fatemeh H. Fard. Humanevalcomm: Benchmarking the communication competence of code generation for llms and llm agent. *ACM Transactions on Software Engineering and Methodology*, 34(7):1–42, 2025. URL <https://arxiv.org/abs/2406.00215>.
- Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, et al. Agent-as-a-judge: Evaluate agents with agents. In *International Conference on Machine Learning*, 2025.